

PROMPT CODEX ĐẦY ĐỦ XÂY DỰNG CRYPTO QUANT EXPERT

FinSight Agent - Binance Spot - Historical + Realtime + ML + API

Phạm vi: Crypto Quant Expert v1

Nguồn dữ liệu: Binance Public Data ZIP, REST Market Data API và WebSocket

Mục tiêu: dùng trực tiếp trong Codex để audit repository và triển khai tuần tự từng phase

Phiên bản tài liệu: 1.0

CÁCH SỬ DỤNG

1. Mở Codex tại thư mục gốc của repository FinSight Agent.
2. Sao chép toàn bộ nội dung prompt từ phần "PROMPT CHÍNH" bên dưới.
3. Ở lần chạy đầu, yêu cầu Codex audit repository và chỉ hoàn thành Phase 1.
4. Sau mỗi phase, kiểm tra PROGRESS.md, test, lint và type check trước khi yêu cầu triển khai phase tiếp theo.
5. Không chạy full backfill 3 năm trong quá trình phát triển; chỉ smoke test với BTCUSDT, ETHUSDT và dữ liệu nhỏ.

PROMPT CHÍNH

Bạn là Senior AI Engineer, Machine Learning Engineer, Data Engineer và Backend Engineer. Hãy kiểm tra repository hiện tại và xây dựng hoàn chỉnh module Crypto Quant Expert cho project FinSight Agent.

I. BỐI CẢNH SẢN PHẨM

Mục tiêu cuối cùng của project là một website phân tích thị trường tài chính:

Người dùng:

1. Chọn loại tài sản:

- Stocks
- Crypto
- Forex

2. Chọn mã:

- Ví dụ BTCUSDT, AAPL, EUR_USD.

3. Chọn chart interval:

- 1m
- 5m
- 15m
- 1h
- 4h
- 1d
- 1w

4. Xem candlestick chart realtime.

5. Nhấn "Nhận định theo interval".

6. Hệ thống tương lai sẽ kết hợp:

- Quant Expert
- News Expert
- Fundamental Expert
- Gating/Fusion Model
- Risk Engine
- Agent Orchestrator

7. Trả về:

- Bullish / Sideways / Bearish
- P(Bullish), P(Sideways), P(Bearish)
- Confidence
- Expected return

- Expected volatility
- Market regime
- Entry zone
- Stop-loss
- Take-profit
- Evidence và explanation.

Trong yêu cầu hiện tại, CHỈ xây dựng hoàn chỉnh Crypto Quant Expert.

Không triển khai News Expert, LLM Agent, Computer Vision hoặc giao dịch tiền thật trong giai đoạn này. Tuy nhiên, thiết kế interface phải đủ sạch để bổ sung các module đó sau này.

Project chỉ phục vụ:

- Research
- Portfolio
- Paper trading
- Model evaluation

Tuyệt đối không:

- Gọi endpoint đặt lệnh.
- Yêu cầu quyền trading.
- Giao dịch bằng tiền thật.
- Hard-code API key.
- Đưa ra cam kết lợi nhuận.

II. NGUYÊN TẮC XỬ LÝ REPOSITORY HIỆN TẠI

Trước khi viết code:

1. Đọc toàn bộ repository hiện tại.
2. Xác định code nào đã có:
 - API crawler
 - Binance client
 - Database
 - Feature engineering
 - Training
 - FastAPI
 - Docker
 - Tests
3. Không viết lại mù quáng code đã chạy đúng.
4. Tái sử dụng và refactor code hiện tại vào kiến trúc mới.
5. Không xóa source code hoặc dữ liệu người dùng nếu chưa thật sự cần thiết.
6. Không xóa file thủ công quan trọng.
7. Các file generated, cache hoặc model cũ phải được nhận diện rõ trước khi xóa.
8. Không để toàn bộ logic trong notebook hoặc một file main.py lớn.
9. Business logic phải nằm trong src/.
10. Script chỉ parse argument và gọi service/pipeline.

Đầu tiên tạo:

docs/quant_expert_gap_report.md

Nội dung báo cáo:

- Cấu trúc repository hiện tại.
- Những phần đã có.
- Những phần có thể tái sử dụng.
- Những lỗi kiến trúc.
- Những module còn thiếu.
- Kế hoạch triển khai theo từng phase.
- Các file dự kiến tạo hoặc chỉnh sửa.

Sau đó mới bắt đầu sửa code.

III. CẤU HÌNH QUANT EXPERT V1

Thị trường:

```
market_type: crypto
exchange: Binance
trading_mode: Spot
quote_asset: USDT
```

Không dùng Futures trong Quant Expert v1.

Universe cố định ban đầu:

REQUIRED_SYMBOLS:

- BTCUSDT
- ETHUSDT

CANDIDATE_SYMBOLS:

- SOLUSDT
- BNBUSDT
- XRPUSDT
- ADAUSDT
- DOGEUSDT
- LINKUSDT

Không được giả định mọi candidate luôn tồn tại.

Mỗi lần build universe phải kiểm tra bằng exchangeInfo:

- status == "TRADING"
- quoteAsset == "USDT"
- isSpotTradingAllowed == true
- Không phải stablecoin/stablecoin pair.
- Không phải leveraged token.
- Có quoteVolume đủ lớn.
- Có đủ dữ liệu lịch sử.
- Không có dữ liệu chất lượng quá thấp.

Nếu candidate không còn hợp lệ:

- Bỏ candidate đó.

- Chọn cặp USDT khác theo quoteVolume.
- Ghi rõ lý do trong universe report.

Universe v1 nên có từ 8 đến 12 cặp.

Ba prediction task chính:

Task 1:

- input_interval: 15m
- forecast_horizon: 1h
- forecast_steps: 4
- model_name: crypto_quant_15m_1h

Task 2:

- input_interval: 1h
- forecast_horizon: 4h
- forecast_steps: 4
- model_name: crypto_quant_1h_4h

Task 3:

- input_interval: 1d
- forecast_horizon: 5d
- forecast_steps: 5
- model_name: crypto_quant_1d_5d

Chart vẫn phải hỗ trợ:

- 1m
- 5m
- 15m
- 1h
- 4h
- 1d
- 1w

Nhưng Quant Expert v1 chỉ dự đoán cho:

- 15m
- 1h
- 1d

Nếu request prediction cho 1m, 5m, 4h hoặc 1w, API phải trả:

```
{
  "status": "model_not_available",
  "supported_prediction_intervals": ["15m", "1h", "1d"]
}
```

Không được âm thầm dùng model sai interval.

IV. NGUỒN DỮ LIỆU VÀ API CHÍNH THỨC

Không scrape giao diện Binance.

Dùng ba cơ chế:

1. Historical bulk:
Binance Public Data ZIP.
2. Historical incremental và reconciliation:
Binance REST Market Data API.
3. Realtime:
Binance WebSocket Market Streams.

4.1. BINANCE PUBLIC DATA ZIP

Base website:

<https://data.binance.vision>

Monthly Spot Kline URL pattern:

<https://data.binance.vision/data/spot/monthly/klines/{SYMBOL}/{INTERVAL}/{SYMBOL}-{INTERVAL}-{YYYY}-{MM}.zip>

Checksum URL:

<https://data.binance.vision/data/spot/monthly/klines/{SYMBOL}/{INTERVAL}/{SYMBOL}-{INTERVAL}-{YYYY}-{MM}.zip.CHECKSUM>

Daily Spot Kline URL pattern:

<https://data.binance.vision/data/spot/daily/klines/{SYMBOL}/{INTERVAL}/{SYMBOL}-{INTERVAL}-{YYYY}-{MM}-{DD}.zip>

Checksum URL:

<https://data.binance.vision/data/spot/daily/klines/{SYMBOL}/{INTERVAL}/{SYMBOL}-{INTERVAL}-{YYYY}-{MM}-{DD}.zip.CHECKSUM>

Chiến lược:

- Monthly ZIP:
Dùng để tải dữ liệu từ ngày bắt đầu đến hết tháng hoàn chỉnh gần nhất.
- REST API:
Dùng để lấy dữ liệu từ sau file monthly cuối cùng đến hiện tại.
- Daily ZIP:
Chỉ là fallback. Không bắt buộc nếu REST hoạt động.

Không tải thủ công bằng browser.

Phải có BulkDownloader tự:

- Sinh URL.
- Tải ZIP.
- Tải CHECKSUM.

- Xác minh SHA-256.
- Retry khi lỗi mạng.
- Skip file đã tải và checksum hợp lệ.
- Tải lại file bị hỏng.
- Ghi ingestion log.
- Giải nén an toàn.
- Không cho zip-slip.
- Hỗ trợ resume.
- Hỗ trợ dry-run.
- Hỗ trợ giới hạn symbol, interval và thời gian.

Lưu ý bắt buộc:

Binance Spot archive từ 01/01/2025 trở đi có timestamp ở microseconds.
Dữ liệu trước đó thường ở milliseconds.

Viết utility tự động phát hiện timestamp unit theo magnitude:

- milliseconds
- microseconds

Sau đó chuyển toàn bộ sang timezone-aware UTC datetime.

Không được luôn dùng unit="ms".

4.2. BINANCE REST MARKET DATA API

REST base URL:

`https://data-api.binance.vision`

Các endpoint public không yêu cầu API key.

A. Kiểm tra kết nối:

GET `/api/v3/ping`

B. Đồng bộ server time:

GET `/api/v3/time`

C. Lấy metadata symbol:

GET `/api/v3/exchangeInfo`

Ví dụ:

GET `https://data-api.binance.vision/api/v3/exchangeInfo?symbol=BTCUSDT`

Các field quan trọng:

- symbol
- status
- baseAsset

- quoteAsset
- baseAssetPrecision
- quoteAssetPrecision
- quotePrecision
- isSpotTradingAllowed
- filters

D. Lấy thống kê thanh khoản 24 giờ:

GET /api/v3/ticker/24hr

Có thể gọi toàn bộ thị trường hoặc theo symbol.

Các field cần dùng:

- symbol
- priceChange
- priceChangePercent
- weightedAvgPrice
- prevClosePrice
- lastPrice
- bidPrice
- askPrice
- openPrice
- highPrice
- lowPrice
- volume
- quoteVolume
- openTime
- closeTime
- count

E. Lấy historical/incremental klines:

GET /api/v3/klines

Parameters:

- symbol
- interval
- startTime
- endTime
- limit

Giới hạn limit phải được xử lý đúng theo tài liệu.
Implement pagination bằng startTime.

Kline response chứa 12 trường theo đúng thứ tự:

0. open_time
1. open
2. high
3. low
4. close

5. base_volume
6. close_time
7. quote_volume
8. trade_count
9. taker_buy_base_volume
10. taker_buy_quote_volume
11. ignore

Không lưu field ignore.

Phải lưu đầy đủ:

- open_time
- close_time
- open
- high
- low
- close
- base_volume
- quote_volume
- trade_count
- taker_buy_base_volume
- taker_buy_quote_volume

F. Các endpoint mở rộng, chưa dùng trực tiếp trong v1 nhưng cần tạo interface:

```
GET /api/v3/aggTrades
GET /api/v3/depth
GET /api/v3/ticker/bookTicker
```

Không tải mass order book hoặc aggTrades trong v1.
Chỉ tạo provider interface và tài liệu để mở rộng sau.

4.3. BINANCE WEBSOCKET

Public market-data WebSocket domain:

`wss://data-stream.binance.vision:443`

Single stream example:

`wss://data-stream.binance.vision:443/ws/btcusdt@kline_15m`

Combined stream example:

`wss://data-stream.binance.vision:443/stream?streams=btcusdt@kline_15m/ethusdt@kline_15m`

Stream symbol phải dùng lowercase.

Quant Expert realtime cần subscribe:

- {symbol}@kline_15m
- {symbol}@kline_1h
- {symbol}@kline_1d

Cho toàn bộ active universe.

WebSocket kline payload phải được parse đầy đủ.

Chỉ coi candle là hoàn chỉnh khi:

`kline["x"] == true`

Nếu `x == false`:

- Có thể lưu trạng thái tạm trong Redis.
- Không ghi thành candle chính thức.
- Không dùng để train.
- Không chạy production inference chính thức.

Khi `x == true`:

1. Normalize.
2. Validate.
3. Upsert candle.
4. Tính feature bằng shared FeatureBuilder.
5. Chọn đúng model interval.
6. Chạy inference.
7. Lưu prediction.
8. Broadcast prediction lên frontend.
9. Lập lịch đánh giá outcome.

WebSocket client phải có:

- Reconnect exponential backoff.
- Ping/pong.
- Logging.
- Connection state.
- Graceful shutdown.
- Idempotent event processing.
- Duplicate protection.
- Maximum retry delay.
- Không crash toàn worker khi một stream lỗi.

Sau reconnect phải REST reconciliation:

1. Đọc open_time cuối trong database.
2. Gọi GET /api/v3/klines.
3. Lấy các closed candle còn thiếu.
4. Validate.
5. Upsert.
6. Chạy inference cho candle chưa xử lý.
7. Tiếp tục WebSocket.

V. CHIẾN LƯỢC LẤY DỮ LIỆU

Không tự động tải dữ liệu cực lớn khi chạy test.

Cần hỗ trợ hai mode:

1. Smoke test:
 - BTCUSDT
 - ETHUSDT
 - 15m
 - 1 đến 3 tháng
2. Full backfill:
 - Active universe 8–12 cặp
 - 15m, 1h, 1d
 - Khoảng 3 năm hoặc toàn bộ lịch sử hợp lệ theo config

Khoảng ngày phải cấu hình động.
Không hard-code ngày kết thúc.

Flow:

```

exchangeInfo + ticker/24hr
  ↓
Universe Builder
  ↓
Monthly ZIP backfill
  ↓
Checksum
  ↓
Raw archive
  ↓
REST incremental backfill
  ↓
Normalize
  ↓
Validate
  ↓
Silver Parquet + candles database
  ↓
Feature engineering
  ↓
Training dataset
  ↓
Model training
  ↓
WebSocket realtime
  ↓
Prediction monitoring
  
```

VI. DATA LAYERS

Xây ba tầng dữ liệu:

```

data/
├── bronze/
  
```

```
├─ silver/
└─ gold/
```

6.1. BRONZE

Giữ dữ liệu gần nguyên bản nhất:

```
data/bronze/binance/spot/klines/
symbol=BTCUSDT/
interval=15m/
year=2025/
month=01/
```

Bao gồm:

- ZIP gốc
- CSV gốc
- CHECKSUM
- Download metadata
- Request metadata

Không sửa raw file.

6.2. SILVER

Dữ liệu đã:

- Normalize schema.
- Chuyển UTC.
- Chuyển numeric type.
- Deduplicate.
- Validate.
- Gắn quality flags.
- Gắn source metadata.

Partition Parquet:

```
data/silver/candles/
exchange=binance/
symbol=BTCUSDT/
interval=15m/
year=2025/
month=01/
```

6.3. GOLD

Dữ liệu dùng cho ML:

```
data/gold/quant_features/
data/gold/training_samples/
data/gold/backtests/
```

Partition theo:

- model task
- dataset version
- symbol
- year hoặc fold khi cần

VII. DATABASE SCHEMA

Dùng PostgreSQL. Nếu project đã có TimescaleDB thì giữ lại.
Nếu chưa có, thiết kế schema tương thích để nâng cấp sang TimescaleDB.

Dùng:

- SQLAlchemy 2 async.
- Alembic migration.
- UTC timestamps.
- Unique constraints.
- Idempotent upsert.

Tạo tối thiểu các bảng:

7.1. instruments

Fields:

- id
- asset_type
- exchange
- symbol
- base_asset
- quote_asset
- status
- is_spot
- is_active
- price_precision
- quantity_precision
- first_available_at
- last_available_at
- metadata JSONB
- created_at
- updated_at

Unique:

(exchange, symbol)

7.2. universe_versions

Fields:

- id
- universe_name

- version
- created_at
- selection_config JSONB
- symbols JSONB
- report JSONB
- is_active

7.3. ingestion_runs

Fields:

- id
- source_type
- provider
- symbol
- interval
- requested_start
- requested_end
- started_at
- completed_at
- status
- downloaded_files
- records_received
- records_inserted
- records_updated
- error_message
- metadata JSONB

source_type:

- monthly_zip
- daily_zip
- rest_backfill
- websocket
- reconciliation

7.4. candles

Fields:

- id
- instrument_id
- interval
- open_time
- close_time
- open
- high
- low
- close
- base_volume
- quote_volume
- trade_count

- taker_buy_base_volume
- taker_buy_quote_volume
- is_closed
- source
- source_file
- quality_status
- quality_flags JSONB
- ingested_at
- updated_at

Unique:

(instrument_id, interval, open_time)

Checks:

- high >= low
- high >= open
- high >= close
- low <= open
- low <= close
- base_volume >= 0
- quote_volume >= 0 khi có giá trị
- trade_count >= 0 khi có giá trị

7.5. data_quality_reports

Fields:

- id
- ingestion_run_id
- instrument_id
- interval
- start_time
- end_time
- total_rows
- duplicate_rows
- missing_candles
- invalid_ohlc_rows
- negative_volume_rows
- null_rows
- timestamp_errors
- quality_score
- details JSONB
- created_at

7.6. quant_features

Fields:

- id
- instrument_id

- interval
- feature_time
- feature_version
- feature_values JSONB hoặc các cột feature chính
- market_regime
- source_candle_time
- data_quality_score
- feature_hash
- created_at

Unique:

(instrument_id, interval, feature_time, feature_version)

7.7. training_samples

Fields:

- id
- instrument_id
- model_task
- input_interval
- forecast_horizon
- feature_time
- feature_version
- dataset_version
- future_return
- normalized_return
- direction_label
- class_weight
- recency_weight
- regime_weight
- quality_weight
- final_weight
- split_name
- fold_id
- created_at

7.8. model_versions

Fields:

- id
- model_name
- expert_type
- asset_type
- model_type
- input_interval
- forecast_horizon
- version
- feature_version
- dataset_version

- parameters JSONB
- metrics JSONB
- artifact_path
- train_start
- train_end
- validation_start
- validation_end
- status
- created_at
- promoted_at

status:

- candidate
- production
- archived
- rejected

7.9. predictions

Fields:

- id
- instrument_id
- model_version_id
- input_interval
- forecast_horizon
- prediction_time
- feature_time
- probability_bullish
- probability_sideways
- probability_bearish
- predicted_class
- confidence
- expected_return
- expected_volatility
- uncertainty
- market_regime
- feature_hash
- trigger_type
- latency_ms
- created_at

trigger_type:

- manual_request
- scheduled
- candle_closed
- reconciliation
- backtest

7.10. prediction_outcomes

Fields:

- id
- prediction_id
- evaluation_time
- entry_price
- exit_price
- actual_return
- normalized_return
- actual_class
- is_correct
- simulated_fee
- simulated_slippage
- net_return
- created_at

VIII. CẤU TRÚC CODE

Ưu tiên modular monolith.

Cấu trúc đề xuất:

```
finsight-agent/
├── apps/
│   ├── api/
│   │   └── main.py
│   └── worker/
│       └── main.py
├── src/
│   ├── finsight/
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── logging.py
│   │   │   ├── exceptions.py
│   │   │   ├── constants.py
│   │   │   └── time.py
│   │   ├── domain/
│   │   │   ├── enums.py
│   │   │   ├── entities.py
│   │   │   ├── schemas.py
│   │   │   └── events.py
│   │   ├── database/
│   │   │   ├── base.py
│   │   │   ├── session.py
│   │   │   ├── models.py
│   │   │   └── repositories/
│   │   │       ├── instrument_repository.py
│   │   │       ├── candle_repository.py
│   │   │       ├── ingestion_repository.py
│   │   │       ├── feature_repository.py
│   │   │       ├── model_repository.py
│   │   │       └── prediction_repository.py
```

```
— providers/
  — market/
    — base.py
    — binance/
      — rest_client.py
      — websocket_client.py
      — public_data_client.py
      — schemas.py
      — normalizer.py
      — timestamp_parser.py

— ingestion/
  — universe_builder.py
  — bulk_downloader.py
  — rest_backfill.py
  — realtime_stream.py
  — reconciliation.py
  — validator.py
  — service.py

— quant/
  — features/
    — price.py
    — volatility.py
    — momentum.py
    — volume.py
    — cross_asset.py
    — time_features.py
    — regime.py
    — registry.py
    — builder.py

  — labels/
    — future_return.py
    — direction.py

  — weighting/
    — class_weight.py
    — recency_weight.py
    — regime_weight.py
    — quality_weight.py
    — combined.py

  — datasets/
    — builder.py

  — splitters/
    — walk_forward.py

  — models/
    — interface.py
    — majority_baseline.py
    — logistic_regression.py
    — lightgbm_model.py
    — calibration.py
    — registry.py

  — training/
    — pipeline.py
    — evaluation.py
    — backtest.py
    — ablation.py
    — report.py

  — inference/
```

```

├── feature_loader.py
├── predictor.py
├── pipeline.py
├── outcome_evaluator.py
├── risk/
│   ├── schemas.py
│   └── basic_risk_engine.py
├── services/
│   ├── market_service.py
│   ├── quant_service.py
│   └── realtime_service.py
├── api/
│   ├── dependencies.py
│   ├── error_handlers.py
│   └── routers/
│       ├── health.py
│       ├── instruments.py
│       ├── market.py
│       ├── quant.py
│       ├── predictions.py
│       └── websocket.py
├── monitoring/
│   ├── metrics.py
│   ├── data_freshness.py
│   ├── prediction_drift.py
│   └── health.py
├── configs/
│   ├── development.yaml
│   ├── ingestion.yaml
│   ├── quant_15m_1h.yaml
│   ├── quant_1h_4h.yaml
│   └── quant_1d_5d.yaml
├── scripts/
│   ├── build_universe.py
│   ├── download_history.py
│   ├── backfill_rest.py
│   ├── validate_market_data.py
│   ├── build_quant_dataset.py
│   ├── train_quant.py
│   ├── backtest_quant.py
│   └── run_market_stream.py
├── tests/
│   ├── unit/
│   ├── integration/
│   └── end_to_end/
├── docs/
├── migrations/
├── data/
├── artifacts/
├── docker-compose.yml
├── pyproject.toml
├── Makefile
├── .env.example
└── README.md

```

Điều chỉnh theo repository hiện tại, không bắt buộc di chuyển vô ích nếu cấu trúc hiện tại đã hợp lý.

IX. FEATURE ENGINEERING

Chỉ sử dụng dữ liệu đã tồn tại tại `feature_time`.

Feature của candle đã đóng tại thời điểm `t` có thể được dùng để dự báo sau `t`.

Không dùng bất kỳ dữ liệu tương lai nào.

Shared FeatureBuilder phải được dùng cho cả:

- Offline training.
- Backtest.
- Realtime inference.

Không viết hai bộ feature code khác nhau.

Các feature v1:

9.1. Price features

- `return_1`
- `return_2`
- `return_4`
- `return_8`
- `return_16`
- `log_return_1`
- `high_low_range`
- `close_open_range`
- `close_position_in_range`
- `distance_from_rolling_high`
- `distance_from_rolling_low`

9.2. Volatility features

- `rolling_volatility_4`
- `rolling_volatility_16`
- `rolling_volatility_48`
- `ATR_14`
- Bollinger bandwidth
- `rolling_return_skew`
- `rolling_return_kurtosis`

Mọi rolling feature phải có `min_periods` hợp lý.

Không fill dữ liệu tương lai.

9.3. Momentum features

- `RSI_14`
- `MACD`
- `MACD signal`

- MACD histogram
- EMA_12
- EMA_26
- EMA ratio
- Rate of Change
- Stochastic oscillator

9.4. Volume và trade activity

- base_volume_change
- quote_volume_change
- quote_volume_ratio_20
- trade_count_change
- trade_count_ratio_20
- average_trade_size
- taker_buy_base_ratio
- taker_buy_quote_ratio
- volume_price_correlation

Công thức:

$$\text{average_trade_size} = \text{base_volume} / \max(\text{trade_count}, \text{epsilon})$$

$$\text{taker_buy_quote_ratio} = \text{taker_buy_quote_volume} / \max(\text{quote_volume}, \text{epsilon})$$

9.5. Cross-asset context

BTCUSDT và ETHUSDT luôn được dùng làm context khi dữ liệu hợp lệ.

Cho mỗi sample:

- btc_return_1
- btc_return_4
- btc_volatility
- eth_return_1
- eth_return_4
- market_median_return
- market_return_breadth
- relative_return_vs_btc
- relative_return_vs_eth

Point-in-time alignment phải chính xác.
Không join candle tương lai.

9.6. Time features

Crypto giao dịch 24/7, nhưng vẫn tạo:

- hour_sin
- hour_cos

- day_of_week_sin
- day_of_week_cos
- weekend_flag

9.7. Market regime

Regime v1 dùng rule-based hoặc clustering đơn giản, nhưng phải tránh leakage.

Rule baseline:

Trend state:

- bullish
- bearish
- neutral

Volatility state:

- low_vol
- high_vol

Kết hợp thành:

- bull_low_vol
- bull_high_vol
- bear_low_vol
- bear_high_vol
- neutral_low_vol
- neutral_high_vol

Threshold phải được fit hoặc tính từ historical rolling distribution.

Không sử dụng toàn bộ future dataset để xác định threshold cho quá khứ.

Sau baseline có thể thử:

- K-Means
- Gaussian Mixture Model
- Hidden Markov Model

Nhưng không bắt buộc trong v1.

X. LABEL GENERATION

Với mỗi model task:

future_return_t =
close[t + forecast_steps] / close[t] - 1

Volatility hiện tại:

sigma_t =
rolling standard deviation của return trước hoặc tại t

Normalized future return:

```
z_t =
future_return_t / max(sigma_t, epsilon)
```

Label:

- $z_t > \text{positive_threshold}$ => BULLISH
- $z_t < \text{negative_threshold}$ => BEARISH
- còn lại => SIDEWAYS

Threshold phải nằm trong config.

Không được tune threshold trên final test set.

Tạo báo cáo:

- Số sample.
- Tỷ lệ BULLISH.
- Tỷ lệ SIDEWAYS.
- Tỷ lệ BEARISH.
- Distribution theo symbol.
- Distribution theo regime.
- Distribution theo thời gian.

Không cố ép ba class bằng nhau một cách giả tạo.

Nếu class imbalance quá lớn, báo cáo và điều chỉnh threshold bằng train/validation.

XI. SAMPLE WEIGHTING

Train model baseline không weight trước.

Sau đó thử riêng:

1. Class weight.
2. Recency weight.
3. Regime weight.
4. Quality weight.
5. Combined weight.

11.1. Class weight

Cân bằng class theo training fold.

Không tính class weight từ validation hoặc test.

11.2. Recency weight

Dữ liệu gần cuối training fold có trọng số cao hơn.

Quan trọng:

Reference time phải là thời điểm cuối training fold.

Không dùng ngày hiện tại để tính recency weight cho mọi historical fold.

11.3. Regime weight

Tăng trọng số cho sample có regime liên quan đến training context.

Phải có config bật/tắt.

Không sử dụng thông tin từ validation/test để xây weight train.

11.4. Quality weight

Giảm trọng số sample:

- Gần data gap.
- Có quality warning.
- Volume bất thường.
- Có dữ liệu bị thiếu.
- Feature không ổn định.

Không forward-fill candle giả rồi coi là giao dịch thật.

11.5. Combined weight

```
final_weight =
class_weight
* recency_weight
* regime_weight
* quality_weight
```

Sau đó:

- Clip min/max.
- Normalize mean weight gần 1.
- Kiểm tra NaN, inf, zero.
- Lưu từng thành phần weight để audit.

XII. TRAINING VÀ VALIDATION

Train một global model cho mỗi task trên nhiều coin.

Ví dụ:

```
BTCUSDT
ETHUSDT
SOLUSDT
```

...

↓

```
crypto_quant_15m_1h
```

Không train riêng model cho từng coin trong baseline.

Có thể làm experiment sau:

- Global model.
- BTC-only model.
- ETH-only model.

Nhưng production baseline là global model theo task.

Các model bắt buộc:

1. Majority Class Baseline.
2. Logistic Regression.
3. LightGBM unweighted.
4. LightGBM weighted.

Không dùng random train_test_split với shuffle=True.

Dùng:

- Time-based split.
- Expanding-window walk-forward validation.
- Final untouched holdout.

Config mặc định hợp lý:

- minimum training window: 12 tháng
- validation window: 3 tháng
- step: 3 tháng
- final holdout: 3 tháng

Nhưng phải tự điều chỉnh nếu dataset ngắn hơn.

Mỗi fold:

1. Fit preprocessing chỉ trên train.
2. Tạo class/recency/regime weight chỉ từ train.
3. Train model.
4. Predict validation.
5. Lưu out-of-fold prediction.
6. Đánh giá.
7. Không thay đổi validation data.

Probability calibration:

- Có thể dùng sigmoid hoặc isotonic.
- Fit calibrator trên validation/calibration period.
- Không fit trên final test.
- Lưu calibrator cùng model artifact.

XIII. METRICS VÀ BACKTEST

Classification metrics:

- Macro-F1
- Weighted-F1
- Balanced accuracy
- Per-class precision
- Per-class recall
- Confusion matrix
- Log loss
- Brier score
- Calibration curve

Analysis:

- Per symbol
- Per interval
- Per market regime
- Per month
- High volatility vs low volatility
- BTC/ETH vs altcoin

Backtest paper simulation:

- Không giao dịch thật.
- Không gọi trading API.
- Không dùng future data để quyết định signal.

Signal baseline:

- BUY nếu $P(\text{BULLISH}) \geq \text{threshold}$.
- SELL/SHORT chỉ dùng cho đánh giá giả lập nếu được bật trong config.
- NO_ACTION nếu confidence thấp.
- Không mặc định giả định có thể short Spot.

Với Spot-safe mode:

- BULLISH => long signal.
- SIDEWAYS => no action.
- BEARISH => exit/cash signal.
- Không short.

Config simulation:

- confidence threshold.
- fee_bps.
- slippage_bps.
- maximum holding horizon.
- position sizing giả lập.

Các giả định phải ghi rõ trong report.

Không mô tả fee/slippage config như mức phí thực tế cố định của Binance.

Trading-oriented metrics:

- Gross return
- Net return
- Sharpe ratio
- Maximum drawdown
- Win rate
- Turnover
- Average trade return
- Number of signals
- Exposure
- Performance sau fee/slippage

Ablation bắt buộc:

M0:

OHLCV cơ bản.

M1:

M0 + trade_count + taker buy.

M2:

M1 + technical features.

M3:

M2 + cross-asset features.

M4:

M3 + class weight.

M5:

M3 + recency weight.

M6:

M3 + regime weight.

M7:

M3 + combined weight.

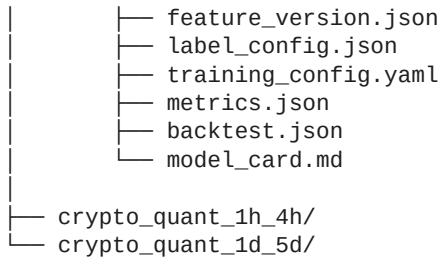
Tạo bảng so sánh đầy đủ.

Không khẳng định weighting cải thiện nếu test không chứng minh.

XIV. MODEL ARTIFACT VÀ REGISTRY

Cấu trúc:

```
artifacts/models/crypto/
├── crypto_quant_15m_1h/
│   └── v1/
│       ├── model.txt
│       ├── calibrator.joblib
│       └── feature_schema.json
```



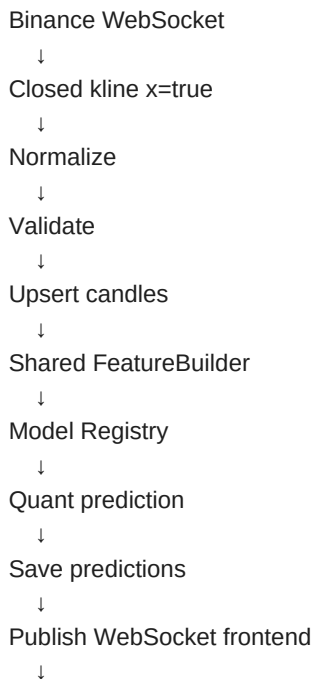
Mỗi model card phải ghi:

- Mục tiêu.
- Universe.
- Input interval.
- Forecast horizon.
- Dataset period.
- Feature list.
- Label definition.
- Split strategy.
- Metrics.
- Performance theo coin.
- Performance theo regime.
- Known limitations.
- Simulation assumptions.
- Không phải financial advice.

Model loader không được load model từ disk trong mỗi request.
Phải cache production model.

XV. REALTIME INFERENCE

Realtime flow:



Schedule outcome evaluation

Inference phải trả:

```
{
  "asset_type": "crypto",
  "exchange": "binance",
  "symbol": "BTCUSD",
  "model_interval": "15m",
  "forecast_horizon": "1h",
  "probabilities": {
    "bullish": 0.61,
    "sideways": 0.26,
    "bearish": 0.13
  },
  "predicted_class": "bullish",
  "confidence": 0.61,
  "expected_return": 0.0042,
  "expected_volatility": 0.0081,
  "market_regime": "bull_high_vol",
  "feature_time": "...",
  "model_version": "crypto_quant_15m_1h_v1",
  "data_freshness_seconds": 4,
  "warnings": []
}
```

Probability phải:

- Nằm trong [0, 1].
- Tổng gần bằng 1.
- Không có NaN/inf.

Khi dữ liệu stale hoặc thiếu:

- Không predict mù.
- Trả status thích hợp.
- Ghi warning.
- Có thể trả NO_ACTION.

XVI. BASIC RISK ENGINE

Quant model không trực tiếp học entry, stop-loss và take-profit.

Tạo một BasicRiskEngine rule-based riêng.

Input:

- Quant prediction.
- Current closed price.
- ATR.
- Volatility.
- Support/resistance đơn giản.
- Input interval.
- Forecast horizon.
- Confidence.

Output:

- signal
- entry_zone
- stop_loss
- take_profit
- risk_reward_ratio
- risk_level
- no_action_reason

Baseline bullish:

- entry quanh latest closed price hoặc EMA/retest zone.
- stop-loss dựa trên ATR multiplier.
- take-profit dựa trên risk/reward multiplier.

Baseline bearish trong Spot-safe mode:

- exit hoặc no_action.
- Không đề xuất short nếu short_mode chưa bật.

Nếu confidence thấp:

- signal = no_action.

Nếu ATR không hợp lệ:

- signal = no_action.

Mọi output phải ghi:

risk_engine_type: "rule_based_v1"

Không mô tả entry/SL/TP là output trực tiếp từ LightGBM.

XVII. FASTAPI CONTRACT

Tạo các endpoint:

17.1. Health

GET /health

Trả:

- API status
- Database status
- Redis status nếu có
- Model registry status
- Last market-data timestamp

17.2. Instruments

GET /api/v1/instruments

Filters:

- asset_type
- exchange
- active

GET /api/v1/instruments/search?query=BTC

17.3. Market history

GET /api/v1/market/history

Query:

- asset_type=crypto
- exchange=binance
- symbol=BTCUSDT
- interval=15m
- from
- to
- limit

Chart intervals hỗ trợ:

- 1m
- 5m
- 15m
- 1h
- 4h
- 1d
- 1w

Nếu database thiếu dữ liệu gần nhất:

- Có thể gọi REST backfill service.
- Không gọi API provider trực tiếp trong router.
- Router gọi MarketService.

17.4. Quant analysis

POST /api/v1/analysis/quant

Request:

```
{
  "asset_type": "crypto",
  "exchange": "binance",
  "symbol": "BTCUSDT",
  "interval": "15m"
}
```

Backend mapping:

15m => forecast horizon 1h
 1h => forecast horizon 4h

1d => forecast horizon 5d

Response:

```
{
  "status": "success",
  "quant": {...},
  "risk": {...}
}
```

Nếu interval chưa có model:

```
{
  "status": "model_not_available",
  "supported_prediction_intervals": ["15m", "1h", "1d"]
}
```

17.5. Prediction history

GET /api/v1/predictions

Filters:

- symbol
- model_name
- interval
- start
- end
- limit

17.6. WebSocket frontend

WS /ws/market/crypto/{symbol}?interval=15m

Events:

- candle_update
- candle_closed
- quant_prediction
- data_warning
- stream_status

XVIII. CLI VÀ COMMAND

Dùng Typer hoặc CLI framework hiện có.

Các command cần có:

1. Build universe

```
python -m finsight.cli universe build \
  --quote-asset USDT \
  --limit 10
```

2. Smoke-test historical download

```
python -m finsight.cli market backfill \
--symbols BTCUSDT,ETHUSDT \
--intervals 15m \
--start 2026-01-01 \
--end 2026-03-31 \
--mode monthly-zip
```

3. Full historical download

```
python -m finsight.cli market backfill \
--universe active \
--intervals 15m,1h,1d \
--years 3 \
--mode hybrid
```

hybrid =
monthly ZIP + REST incremental.

4. Validate

```
python -m finsight.cli market validate \
--universe active \
--intervals 15m,1h,1d
```

5. Build dataset

```
python -m finsight.cli quant build-dataset \
--config configs/quant_15m_1h.yaml
```

6. Train

```
python -m finsight.cli quant train \
--config configs/quant_15m_1h.yaml
```

7. Backtest

```
python -m finsight.cli quant backtest \
--model crypto_quant_15m_1h \
--version v1
```

8. Run realtime stream

```
python -m finsight.cli market stream \
--universe active \
--intervals 15m,1h,1d
```

9. Run API

```
uvicorn apps.api.main:app --reload
```

Các command phải:

- Có help.
- Có validation.

- Có logging.
- Trả exit code đúng.
- Không swallow exception.
- Hỗ trợ dry-run khi phù hợp.

XIX. CONFIG FILE

Tạo configs/quant_15m_1h.yaml:

```
model_name: crypto_quant_15m_1h
asset_type: crypto
exchange: binance
market_type: spot
input_interval: 15m
forecast_horizon: 1h
forecast_steps: 4
```

```
universe:
  name: crypto_spot_usdt_v1
  min_symbols: 8
  max_symbols: 12
  required_symbols:
    - BTCUSDT
    - ETHUSDT
```

```
data:
  history_years: 3
  source: hybrid
  use_monthly_zip: true
  use_rest_incremental: true
  validate_checksum: true
```

```
features:
  version: crypto_quant_features_v1
  include_price: true
  include_volatility: true
  include_momentum: true
  include_volume: true
  include_cross_asset: true
  include_time: true
  include_regime: true
```

```
labels:
  type: volatility_adjusted_direction
  positive_threshold: 0.5
  negative_threshold: -0.5
  volatility_window: 48
```

```
weights:
  class_weight: true
  recency_weight: false
  regime_weight: false
  quality_weight: true
  min_weight: 0.05
```

max_weight: 5.0

```
validation:
  type: expanding_walk_forward
  min_train_months: 12
  validation_months: 3
  step_months: 3
  final_holdout_months: 3
```

```
model:
  type: lightgbm
  objective: multiclass
  random_seed: 42
```

```
backtest:
  spot_safe_mode: true
  probability_threshold: 0.55
  fee_bps: 10
  slippage_bps: 5
```

Các con số fee/slippage chỉ là simulation config, không phải tuyên bố mức phí thực tế.

Tạo config tương tự cho:

- quant_1h_4h.yaml
- quant_1d_5d.yaml

XX. TESTING

Dùng pytest.

Unit tests bắt buộc:

1. Binance URL builder.
2. Checksum verification.
3. Safe ZIP extraction.
4. Millisecond timestamp parsing.
5. Microsecond timestamp parsing.
6. Kline REST response parser.
7. WebSocket kline parser.
8. Candle closed flag x=true.
9. Candle validation.
10. Duplicate detection.
11. Missing-candle detection.
12. Idempotent upsert.
13. Feature calculation.
14. Rolling feature không dùng future.
15. Cross-asset point-in-time alignment.
16. Future return label.
17. Class assignment.
18. Recency weight tính theo cuối fold train.
19. Combined weight normalization.

- 20. Walk-forward split không overlap sai.
- 21. Model probability sum.
- 22. Model registry load/cache.
- 23. Model interval mapping.
- 24. API response schema.
- 25. Missing model response.
- 26. Reconciliation sau stream gap.
- 27. Prediction outcome evaluation.
- 28. Risk engine no-action khi confidence thấp.

Integration tests:

- Mock Binance REST server.
- Historical backfill flow.
- Database upsert flow.
- Dataset build flow.
- Train small fixture model.
- Inference flow.
- Realtime closed-candle flow.
- API + database flow.

End-to-end test:

Fixture nhỏ:

BTCUSDT + ETHUSDT

15m

vài tháng synthetic hoặc committed fixture nhỏ

Flow:

ingest

- validate
- features
- labels
- train
- save model
- load model
- predict
- store prediction
- evaluate outcome

Unit test không được phụ thuộc mạng internet.

Network calls phải mock.

XXI. MONITORING

Theo dõi tối thiểu:

- REST request count.
- REST error count.
- HTTP 429 count.

- Downloaded file count.
- Checksum failure count.
- WebSocket connected status.
- WebSocket reconnect count.
- Last candle timestamp.
- Data freshness.
- Missing candle count.
- Inference latency.
- Prediction count.
- Prediction distribution.
- Model version.
- Outcome accuracy.
- Feature drift cơ bản.
- Prediction drift cơ bản.

Nếu dùng Prometheus hiện có, thêm metrics.
Nếu chưa có, tạo abstraction và logging trước.

XXII. DOCKER VÀ ENVIRONMENT

Ưu tiên:

- Python 3.12
- FastAPI
- Pydantic v2
- SQLAlchemy 2 async
- Alembic
- PostgreSQL/TimescaleDB
- Redis cho realtime state/pubsub nếu project đã dùng
- httpx
- websockets
- tenacity
- pandas hoặc Polars
- PyArrow
- NumPy
- scikit-learn
- LightGBM
- joblib
- Typer
- pytest
- Ruff
- mypy hoặc pyright

Tạo:

- docker-compose.yml
- API Dockerfile
- Worker Dockerfile
- .env.example

Không cần API key cho Binance public market data.

.env.example không chứa secret thật.

XXIII. TÀI LIỆU

Cập nhật README.md với:

1. Project overview.
2. Quant Expert architecture.
3. Data sources.
4. Exact Binance endpoints.
5. Data flow.
6. Database schema.
7. Feature definitions.
8. Label definition.
9. Model tasks.
10. Walk-forward strategy.
11. Commands.
12. API endpoints.
13. Realtime flow.
14. Testing.
15. Docker startup.
16. Known limitations.
17. No-financial-advice disclaimer.

Tạo thêm:

```
docs/api_sources.md
docs/data_dictionary.md
docs/feature_dictionary.md
docs/labeling_and_leakage.md
docs/walk_forward_validation.md
docs/realtime_reconciliation.md
docs/quant_expert_architecture.md
docs/runbook.md
```

Tạo model card cho mỗi trained model.

XXIV. PHASE TRIỂN KHAI

Triển khai tuần tự.

PHASE 0 – Audit repository

Deliverables:

- gap report.
- implementation plan.
- file-change plan.

PHASE 1 – Binance provider và universe

Deliverables:

- exchangeInfo client.

- ticker/24hr client.
- Universe Builder.
- instruments table.
- unit tests.
- universe report.

Definition of Done:

- BTCUSDT và ETHUSDT được giữ nếu valid.
- Có universe 8–12 symbol.
- Mọi symbol được kiểm tra runtime.

PHASE 2 – Historical ingestion

Deliverables:

- Monthly ZIP downloader.
- Checksum.
- Timestamp normalization.
- REST incremental downloader.
- Bronze/Silver storage.
- ingestion_runs.
- candles.
- validation report.

Definition of Done:

- Tải smoke test BTCUSDT/ETHUSDT thành công.
- Không duplicate.
- Timestamp UTC đúng.
- Xử lý được ms và μ s.
- Có thể resume.

PHASE 3 – Feature và dataset

Deliverables:

- Shared FeatureBuilder.
- Regime baseline.
- Label Builder.
- Weight Builder.
- Gold Parquet.
- Dataset report.

Definition of Done:

- Dataset tái tạo bằng một command.
- Không có leakage rõ ràng.
- Có class distribution report.

PHASE 4 – Training và backtest

Deliverables:

- Majority baseline.
- Logistic Regression.
- LightGBM unweighted.
- LightGBM weighted.
- Walk-forward.
- Calibration.
- Ablation.

- Backtest.
- Model card.

Definition of Done:

- Có bảng metric.
- Có final untouched holdout.
- Có artifact save/load.
- Không khẳng định cải thiện nếu kết quả không hỗ trợ.

PHASE 5 — Realtime

Deliverables:

- Combined WebSocket streams.
- x=true handling.
- Reconnect.
- REST reconciliation.
- Realtime features.
- Realtime inference.
- Prediction storage.
- Outcome scheduling.

Definition of Done:

- Closed candle tự tạo prediction.
- Stream mất kết nối có thể backfill phần thiếu.
- Không tạo duplicate prediction.

PHASE 6 — API và Risk Engine

Deliverables:

- FastAPI endpoints.
- Frontend WebSocket.
- Basic Risk Engine.
- Data freshness warning.
- Prediction history.

Definition of Done:

- Request BTCUSDT + 15m trả Quant prediction cho horizon 1h.
- Request unsupported interval trả model_not_available.
- Entry/SL/TP được ghi rõ là rule-based.

PHASE 7 — Hardening

Deliverables:

- Tests.
- Monitoring.
- Docker.
- Documentation.
- Runbook.
- CI.

XXV. QUY TẮC CHẤT LƯỢNG

1. Không tạo code giả chỉ có pass/TODO.

2. Không trả dữ liệu mock ở production endpoint.
3. Không hard-code current date.
4. Không hard-code symbol validity.
5. Không dùng random split.
6. Không fit scaler trên toàn bộ dataset.
7. Không tính feature bằng future data.
8. Không dùng final test để tune threshold.
9. Không forward-fill candle mất rồi coi như giao dịch thật.
10. Không load model mỗi request.
11. Không train model mỗi candle.
12. Không tự động download full dataset khi chạy unit test.
13. Không commit raw data, model lớn hoặc secret.
14. Mọi operation phải idempotent.
15. Mọi model phải có version.
16. Mọi feature set phải có version.
17. Mọi dataset phải có version.
18. Mọi prediction phải lưu feature_time và model_version.
19. Mọi inference phải có data freshness.
20. Mọi API error phải có schema rõ ràng.

XXVI. CÁCH LÀM VIỆC VÀ BÁO CÁO

Không chỉ đưa ra kế hoạch. Hãy thực sự sửa và tạo code.

Trình tự:

1. Audit repository.
2. Viết gap report.
3. Triển khai từng phase.
4. Sau mỗi phase:
 - Chạy formatter.
 - Chạy lint.
 - Chạy type check nếu có.
 - Chạy tests.
 - Ghi file đã thay đổi.
 - Ghi command đã chạy.
 - Ghi kết quả.
 - Ghi lỗi hoặc hạn chế còn lại.
 - Cập nhật PROGRESS.md.

Không tự động chạy full three-year download.

Chỉ chạy smoke test nhỏ trong quá trình phát triển.

Tạo command rõ ràng để người dùng tự chạy full backfill sau.

Chỉ hỏi lại người dùng khi:

- Cần xóa dữ liệu quan trọng.
- Cần credential không thể tránh.
- Có lựa chọn kiến trúc ảnh hưởng lớn mà repository hiện tại không cung cấp đủ thông tin.

Trong các trường hợp còn lại, chọn default hợp lý, ghi rõ assumption và tiếp tục.

XXVII. KẾT QUẢ CUỐI CÙNG PHẢI CÓ

Repository sau khi hoàn thành phải:

1. Tự xây universe từ Binance.
2. Tự tải historical monthly ZIP.
3. Tự kiểm tra checksum.
4. Tự xử lý millisecond/microsecond timestamps.
5. Tự REST backfill phần mới.
6. Tự kiểm tra chất lượng candle.
7. Tự build feature.
8. Tự build label.
9. Tự build sample weight.
10. Train được ba Quant model task.
11. Có walk-forward evaluation.
12. Có backtest paper simulation.
13. Có model registry.
14. Có realtime Binance WebSocket.
15. Có REST reconciliation.
16. Có FastAPI inference.
17. Có WebSocket frontend.
18. Có prediction outcomes.
19. Có basic risk output.
20. Có tests, Docker, README và model card.

Không xây News Expert trong lần này, nhưng Quant output schema phải sẵn sàng để sau này Fusion/Gating Model có thể nhận:

```
{
  "expert_name": "quant",
  "asset_type": "crypto",
  "symbol": "BTCUSD",
  "interval": "15m",
  "forecast_horizon": "1h",
  "probabilities": {
    "bullish": 0.61,
    "sideways": 0.26,
    "bearish": 0.13
  },
  "confidence": 0.61,
  "uncertainty": 0.18,
  "freshness": 0.99,
  "available": true,
  "model_version": "crypto_quant_15m_1h_v1"
}
```

Hãy bắt đầu bằng việc audit repository hiện tại và tạo docs/quant_expert_gap_report.md, sau đó triển khai PHASE 1.

PROMPT TIẾP TỤC SAU MỖI PHASE

Tiếp tục triển khai phase tiếp theo theo đúng prompt gốc. Trước khi sửa code, hãy đọc PROGRESS.md và quant_expert_gap_report.md, kiểm tra lại kết quả phase trước, sửa các lỗi còn tồn tại, sau đó triển khai phase mới. Chạy đầy đủ test, lint và type check trước khi kết thúc.

Lưu ý triển khai

- Ưu tiên hoàn thành một phase chạy thật thay vì tạo nhiều module chỉ có TODO hoặc pass.
- Không sử dụng trading endpoint và không giao dịch tiền thật.
- Các số liệu fee/slippage chỉ là giả lập trong backtest, không phải cam kết mức phí thực tế.
- Quant output phải giữ schema ổn định để nối News Expert, Gating/Fusion, Risk Engine và Agent ở các giai đoạn sau.